2. When an end receives a packet containing data, but has no data to send, it needs to acknowledge the receipt of the packet within a specified time (usually 500 ms).
3. An end must send at least one SACK for every other packet it receives. This rule overrides the second rule.
4. When a packet arrives with out-of-order data chunks, the receiver needs to immediately send a SACK chunk reporting the situation to the sender.
5. When an end receives a packet with duplicate DATA chunks and no new DATA chunks, the duplicate data chunks must be reported immediately with a SACK chunk.

Congestion Control

SCTP, like TCP, is a transport-layer protocol with packets subject to congestion in the network. The SCTP designers have used the same strategies for congestion control as those used in TCP.

8.5 QUALITY OF SERVICE

The Internet was originally designed for best-effort service with of guarantee of predictable performance. Best-effort service is often sufficient for a traffic that is not sensitive to delay, such as file transfers and e-mail. Such a traffic is called *elastic* because it can stretch to work under delay conditions; it is also called *available bit rate* because applications can speed up or slow down according to the available bit rate.

The real-time traffic generated by some multimedia applications. The real-time traffic is delay sensitive and therefore requires guaranteed and predictive performance.

Quality of service (**QoS**) is an internetworking issue that refers to a set of techniques and mechanisms that guarantees the performance of the network to deliver predictable service to an application program.

8.5.1 Data-Flow Characteristics

If we want to provide quality of service for an Internet application, we first need to define what we need for each application. Traditionally, four types of characteristics are attributed to a flow: *reliability, delay, jitter,* and *bandwidth*. Let us first define these characteristics and then investigate the requirements of each application type.

Definitions

We can give informal definitions of the above four characteristics:

- ***Reliability.*** *Reliability* is a characteristic that a flow needs in order to deliver the packets safe and sound to the destination. Lack of reliability means losing a packet or acknowledgment, which entails retransmission. However, the sensitivity of different application programs to reliability varies. For example, reliable transmission is more important for electronic mail, file transfer, and Internet access than for telephony or audio conferencing.
- ***Delay.*** Source-to-destination *delay* is another flow characteristic. Again, applications can tolerate delay in different degrees. In this case, telephony, audio conferencing, video conferencing, and remote login need minimum delay, while delay in file transfer or e-mail is less important.

- ***Jitter.*** *Jitter* is the variation in delay for packets belonging to the same flow. For example, if four packets depart at times 0, 1, 2, 3 and arrive at 20, 21, 22, 23, all have the same delay, 20 units of time. On the other hand, if the above four packets arrive at 21, 23, 24, and 28, they will have different delays. For applications such as audio and video, the first case is completely acceptable; the second case is not. For these applications, it does not matter if the packets arrive with a short or long delay as long as the delay is the same for all packets. These types of applications do not tolerate jitter.
- ***Bandwidth.*** Different applications need different bandwidths. In video conferencing we need to send millions of bits per second to refresh a color screen while the total number of bits in an e-mail may not reach even a million.

Sensitivity of Applications

Now let us see how various applications are sensitive to some flow characteristics. Table 8.9 gives a summary of applications types and their sensitivity.

Table 8.9 *Sensitivity of applications to flow characteristics*

Application	*Reliability*	*Delay*	*Jitter*	*Bandwidth*
FTP	High	Low	Low	Medium
HTTP	High	Medium	Low	Medium
Audio-on-demand	Low	Low	High	Medium
Video-on-demand	Low	Low	High	High
Voice over IP	Low	High	High	Low
Video over IP	Low	High	High	High

For those applications with a high level of sensitivity to reliability, we need to do error checking and discard the packet if corrupted. For those applications with a high level of sensitivity to delay, we need to be sure that they are given priority in transmission. For those applications with a high level of sensitivity to jitter, we need to be sure that the packets belonging to the same application pass the network with the same delay. Finally, for those applications that require high bandwidth, we need to allocate enough bandwidth to be sure that the packets are not lost.

8.5.2 Flow Classes

Based on the flow characteristics, we can classify flows into groups, with each group having the required level of each characteristic. The Internet community has not yet defined such a classification formally. However, we know, for example, that a protocol like FTP needs a high level of reliability and probably a medium level of bandwidth, but the level of delay and jitter is not important for this protocol.

Example 8.11

Although the Internet has not defined flow classes formally, the ATM protocol does. As per ATM specifications, there are five classes of defined service.

a. ***Constant Bit Rate (CBR).*** This class is used for emulating circuit switching. CBR applications are quite sensitive to cell-delay variation. Examples of CBR are telephone traffic, video conferencing, and television.

b. ***Variable Bit Rate-Non Real Time (VBR-NRT).*** Users in this class can send traffic at a rate that varies with time depending on the availability of user information. An example is multimedia e-mail.

c. ***Variable Bit Rate-Real Time (VBR-RT).*** This class is similar to VBR–NRT but is designed for applications such as interactive compressed video that are sensitive to cell-delay variation.

d. ***Available Bit Rate (ABR).*** This class of ATM services provides rate-based flow control and is aimed at data traffic such as file transfer and e-mail.

e. ***Unspecified Bit Rate (UBR).*** This class includes all other classes and is widely used today for TCP/IP.

8.5.3 Flow Control to Improve QoS

Although formal classes of flow are not defined in the Internet, an IP datagram has a ToS field that can informally define the type of service required for a set of datagrams sent by an application. If we assign a certain type of application a single level of required service, we can then define some provisions for those levels of service. These can be done using several mechanisms.

Scheduling

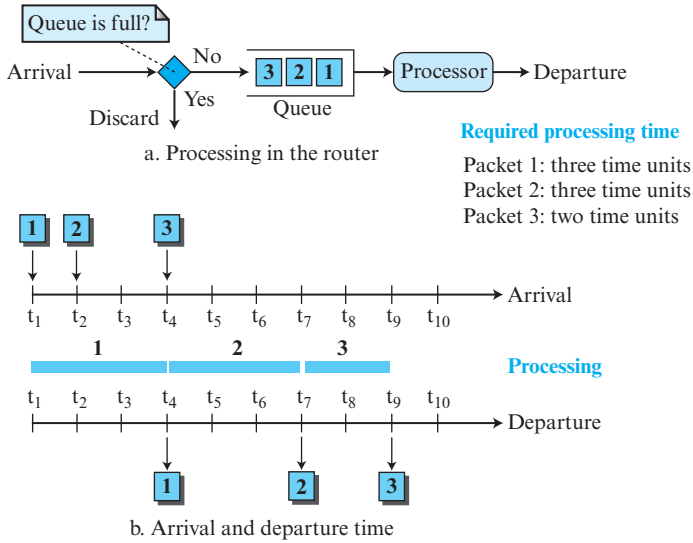
Treating packets (datagrams) in the Internet based on their required level of service can mostly happen at the routers. It is at a router that a packet may be delayed, suffer from jitters, be lost, or be assigned the required bandwidth. A good scheduling technique treats the different flows in a fair and appropriate manner. Several scheduling techniques are designed to improve the quality of service. We discuss three of them here: *FIFO queuing*, *priority queuing,* and *weighted fair queuing*.

FIFO Queuing

In **first-in, first-out (FIFO) queuing,** packets wait in a buffer (queue) until the node (router) is ready to process them. If the average arrival rate is higher than the average processing rate, the queue will fill up and new packets will be discarded. A FIFO queue is familiar to those who have had to wait for a bus at a bus stop. Figure 8.59 shows a conceptual view of a FIFO queue. The figure also shows the timing relationship between arrival and departure of packets in this queuing. Packets from different applications (with different sizes) arrive at the queue, are processed, and depart. A larger packet definitely may need a longer processing time. In the figure, packets 1 and 2 need three time units of processing, but packet 3, which is smaller, needs two time units. This means that packets may arrive with some delays but depart with different delays. If the packets belong to the same application, this produces jitters. If the packets belong to different applications, this also produces jitters for each application.

FIFO queuing is the default scheduling in the Internet. The only thing that is guaranteed in this type of queuing is that the packets depart in the order they arrive. Does FIFO queuing distinguish between packet classes? The answer is definitely no. This type of queuing is what we will see in the Internet with no service differentiation between packets from different sources. With FIFO queuing, all packets are treated the same in a packet-switched network. No matter if a packet belongs to FTP, or Voice over IP, or an e-mail message, they will be equally subject to loss, delay, and jitter. The bandwidth allocated for each application depends on how many packets arrive at the

Figure 8.59 *FIFO queue*



router in a period of time. If we need to provide different services to different classes of packets, we need to have other scheduling mechanisms.

Priority Queuing

Queuing delay in FIFO queuing often degrades quality of service in the network. A frame carrying real-time packets may have to wait a long time behind a frame carrying a small file. We solve this problem using multiple queues and priority queuing.

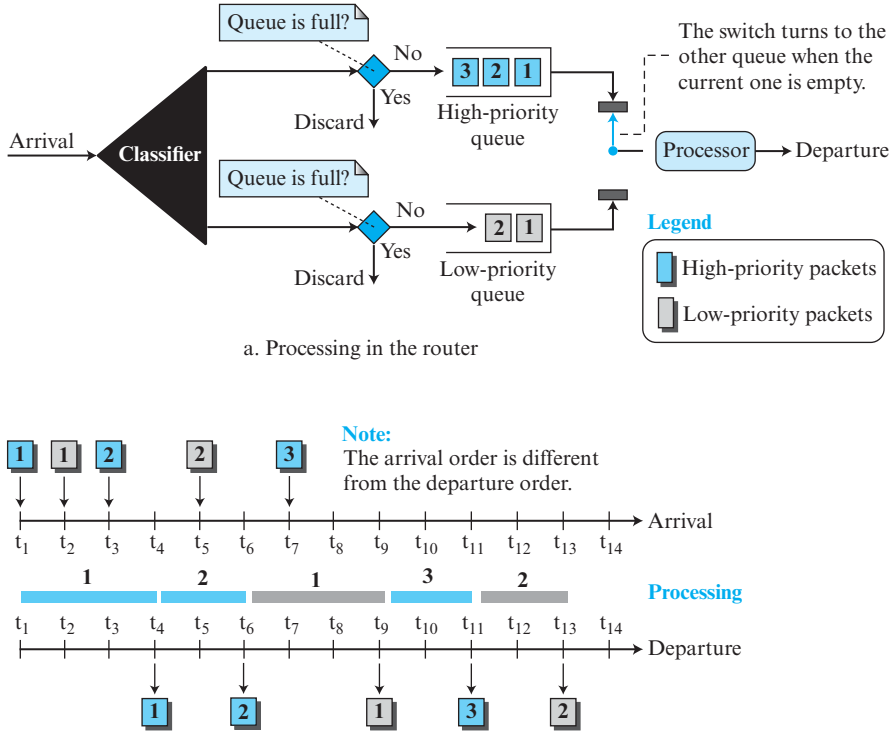
In **priority queuing,** packets are first assigned to a priority class. Each priority class has its own queue. The packets in the highest-priority queue are processed first. Packets in the lowest-priority queue are processed last. Note that the system does not stop serving a queue until it is empty. A packet priority is determined from a specific field in the packet header: the ToS field of an IPv4 header, the priority field of IPv6, a priority number assigned to a destination address, or a priority number assigned to an application (destination port number), and so on.

Figure 8.60 shows priority queuing with two priority levels (for simplicity). A priority queue can provide better QoS than the FIFO queue because higher-priority traffic, such as multimedia, can reach the destination with less delay. However, there is a potential drawback. If there is a continuous flow in a high-priority queue, the packets in the lower-priority queues will never have a chance to be processed. This is a condition called *starvation*. Severe starvation may result in dropping of some packets of lower priority. In the figure, the packets of higher priority are sent out before the packets of lower priority.

Weighted Fair Queuing

A better scheduling method is **weighted fair queuing.** In this technique, the packets are still assigned to different classes and admitted to different queues. The queues,

Figure 8.60 *Priority queuing*

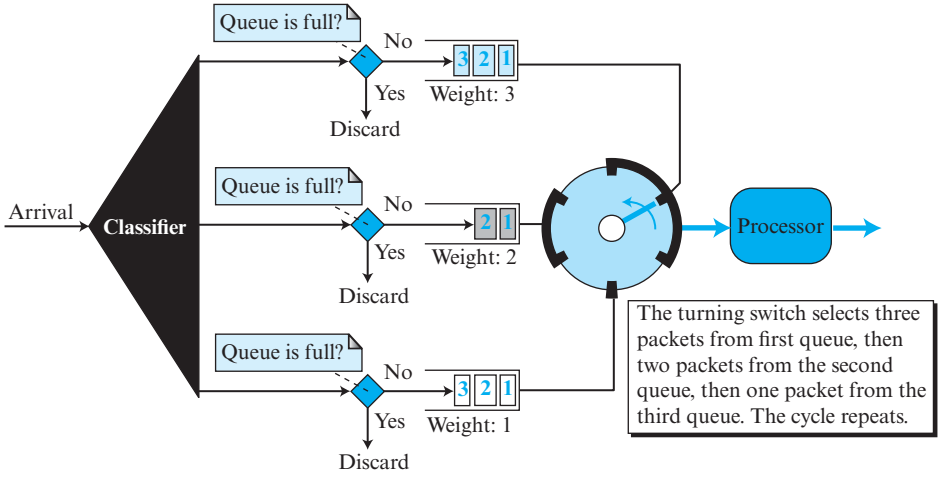


however, are weighted based on the priority of the queues; higher priority means a higher weight. The system processes packets in each queue in a round-robin fashion with the number of packets selected from each queue based on the corresponding weight. For example, if the weights are 3, 2, and 1, three packets are processed from the first queue, two from the second queue, and one from the third queue. In this way, we have fair queuing with priority. Figure 8.61 shows the technique with three classes. In weighted fair queuing, each class may receive a small amount of time in each time period. In other words, a fraction of time is devoted to serve each class of packets, but the fraction depends on the priority of the class. For example, in the figure, if the throughput for the router is R, the class with the highest priority may have the throughput of R/2, the middle class may have the throughput of R/3, and the class with the lowest priority may have the throughput of R/6. However, this situation is true if all three classes have the same packet size, which may not occur. Packets of different sizes may create many imbalances in dividing a decent share of time between different classes.

Traffic Shaping or Policing

To control the amount and the rate of traffic is called *traffic shaping* or *traffic policing*. The first term is used when the traffic leaves a network; the second term is used when

Figure 8.61 *Weighted fair queuing*

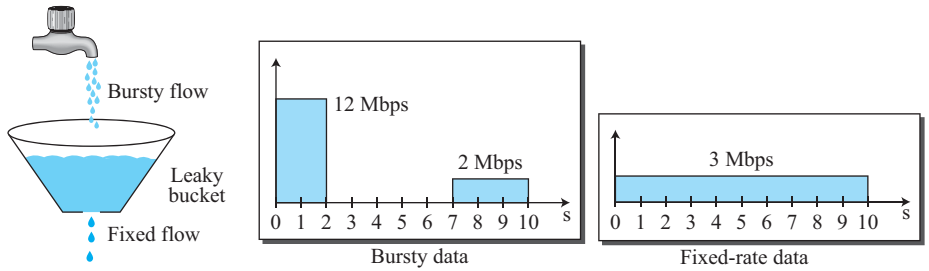


the data enters the network. Two techniques can shape or police the traffic: leaky bucket and token bucket.

Leaky Bucket

If a bucket has a small hole at the bottom, the water leaks from the bucket at a constant rate as long as there is water in the bucket. The rate at which the water leaks does not depend on the rate at which the water is input unless the bucket is empty. If the bucket is full, the water overflows. The input rate can vary, but the output rate remains constant. Similarly, in networking, a technique called **leaky bucket** can smooth out bursty traffic. Bursty chunks are stored in the bucket and sent out at an average rate. Figure 8.62 shows a leaky bucket and its effects.

Figure 8.62 *Leaky bucket*

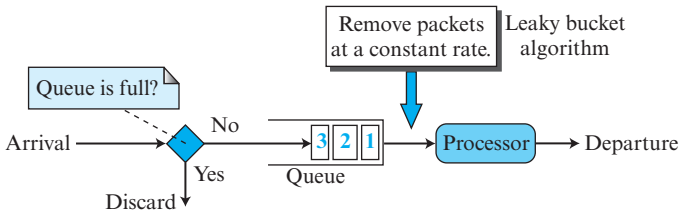


In the figure, we assume that the network has committed a bandwidth of 3 Mbps for a host. The use of the leaky bucket shapes the input traffic to make it conform to this commitment. In Figure 8.62 the host sends a burst of data at a rate of 12 Mbps for 2 seconds, for a total of 24 Mb of data. The host is silent for 5 seconds and then sends

data at a rate of 2 Mbps for 3 seconds, for a total of 6 Mb of data. In all, the host has sent 30 Mb of data in 10 seconds. The leaky bucket smooths the traffic by sending out data at a rate of 3 Mbps during the same 10 seconds. Without the leaky bucket, the beginning burst may have hurt the network by consuming more bandwidth than is set aside for this host. We can also see that the leaky bucket may prevent congestion.

A simple leaky bucket implementation is shown in Figure 8.63. A FIFO queue holds the packets. If the traffic consists of fixed-size packets (e.g., cells in ATM networks), the process removes a fixed number of packets from the queue at each tick of the clock. If the traffic consists of variable-length packets, the fixed output rate must be based on the number of bytes or bits.

Figure 8.63 *Leaky bucket implementation*



The following is an algorithm for variable-length packets:

1. Initialize a counter to n at the tick of the clock.
2. If n is greater than the size of the packet, send the packet and decrement the counter by the packet size. Repeat this step until the counter value is smaller than the packet size.
3. Reset the counter to n and go to step 1.

A leaky bucket algorithm shapes bursty traffic into fixed-rate traffic by averaging the data rate. It may drop the packets if the bucket is full.

Token Bucket

The leaky bucket is very restrictive. It does not credit an idle host. For example, if a host is not sending for a while, its bucket becomes empty. Now if the host has bursty data, the leaky bucket allows only an average rate. The time when the host was idle is not taken into account. On the other hand, the **token bucket** algorithm allows idle hosts to accumulate credit for the future in the form of tokens.

Assume the capacity of the bucket is c tokens and tokens enter the bucket at the rate of r token per second. The system removes one token for every cell of data sent. The maximum number of cells that can enter the network during any time interval of length t is shown below.

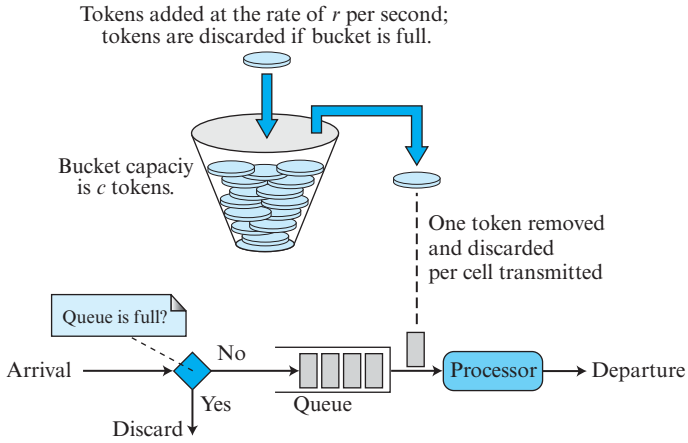
$$\textbf{Maximum number of packets} = rt + c$$

The maximum average rate for the token bucket is shown below.

$$\textbf{Maximum average rate} = (rt + c)/t \textbf{ packets per second}$$

This means that the token bucket limits the average packet rate to the network. Figure 8.64 shows the idea.

Figure 8.64 *Token bucket*



Example 8.12

Let assume that the bucket capacity is 10,000 tokens and tokens are added at the rate of 1000 tokens per second. If the system is idle for 10 seconds (or more), the bucket collects 10,000 tokens and becomes full. Any additional tokens will be discarded. The maximum average rate is shown below.

$$\textbf{Maximum average rate} = (1000t + 10{,}000)/t$$

The token bucket can easily be implemented with a counter. The token is initialized to zero. Each time a token is added, the counter is incremented by 1. Each time a unit of data is sent, the counter is decremented by 1. When the counter is zero, the host cannot send data.

The token bucket allows bursty traffic at a regulated maximum rate.

Combining Token Bucket and Leaky Bucket

The two techniques can be combined to credit an idle host and at the same time regulate the traffic. The leaky bucket is applied after the token bucket; the rate of the leaky bucket needs to be higher than the rate of tokens dropped in the bucket.

Resource Reservation

A flow of data needs resources such as a buffer, bandwidth, CPU time, and so on. The quality of service is improved if these resources are reserved beforehand. Below, we discuss a QoS model called Integrated Services, which depends heavily on resource reservation to improve the quality of service.

Admission Control

Admission control refers to the mechanism used by a router or a switch to accept or reject a flow based on predefined parameters called *flow specifications*. Before a router accepts a flow for processing, it checks the flow specifications to see if its capacity can handle the new flow. It takes into account bandwidth, buffer size, CPU speed, etc., as well as its previous commitments to other flows. Admission control in ATM networks is known as *Connection Admission Control* (CAC), which is a major part of the strategy for controlling congestion.

8.5.4 Integrated Services (IntServ)

Traditional Internet provided only the best-effort delivery service to all users regardless of what was needed. Some applications, however, needed a minimum amount of bandwidth to function (such as real-time audio and video). To provide different QoS for different applications, IETF (discussed in Chapter 1) developed the **integrated services** (**IntServ**) model. In this model, which is a *flow-based* architecture, resources such as bandwidth are explicitly reserved for a given data flow. In other words, the model is considered a specific requirement of an application in one particular case regardless of the application type (data transfer, or voice over IP, or video-on-demand). What is important are the resources the application needs, not what the application is doing.

The model is based on three schemes:

1. The packets are first classified according to the service they require.
2. The model uses scheduling to forward the packets according to their flow characteristics.
3. Devices like routers use *admission control* to determine if the device has the capability (available resources to handle the flow) before making a commitment. For example, if an application requires a very high data rate, but a router in the path cannot provide such a data rate, it denies the admission.

Before we discuss this model, we need to emphasize that the model is flow-based, which means that all accommodations need to be made before a flow can start. This implies that we need a connection-oriented service at the network layer. A connection establishment phase is needed to inform all routers of the requirement and get their approval (admission control). However, since IP is currently a connectionless protocol, we need another protocol to be run on top of IP to make it a connection-oriented protocol before we can use this model. This protocol is called **Resource Reservation Protocol (RSVP)** and will be discussed shortly.

Integrated Services is a flow-based QoS model designed for IP.
In this model packets are marked by routers according to flow characteristics.

Flow Specification

We said that IntServ is flow-based. To define a specific flow, a source needs to define a *flow specification*, which is made of two parts:

1. ***Rspec (resource specification).*** Rspec defines the resource that the flow needs to reserve (buffer, bandwidth, etc.).

2. ***Tspec (traffic specification).*** Tspec defines the traffic characterization of the flow, which we discussed before.

Admission

After a router receives the flow specification from an application, it decides to admit or deny the service. The decision is based on the previous commitments of the router and the current availability of the resource.

Service Classes

Two classes of services have been defined for Integrated Services: guaranteed service and controlled-load service.

Guaranteed Service Class

This type of service is designed for real-time traffic that needs a guaranteed minimum end-to-end delay. The end-to-end delay is the sum of the delays in the routers, the propagation delay in the media, and the setup mechanism. Only the first, the sum of the delays in the routers, can be guaranteed by the router. This type of service guarantees that the packets will arrive within a certain delivery time and are not discarded if flow traffic stays within the boundary of *Tspec*. We can say that guaranteed services are *quantitative services*, in which the amount of end-to-end delay and the data rate must be defined by the application. Normally guaranteed services are required for real-time applications (voice over IP).

Controlled-Load Service Class

This type of service is designed for applications that can accept some delays but are sensitive to an overloaded network and to the danger of losing packets. Good examples of these types of applications are file transfer, e-mail, and Internet access. The controlled-load service is a *qualitative service* in that the application requests the possibility of low-loss or no-loss packets.

Resource Reservation Protocol (RSVP)

We said that the Integrated Services model needs a connection-oriented network layer. Since IP is a connectionless protocol, a new protocol is designed to run on top of IP to make it connection-oriented. A connection-oriented protocol needs to have connection establishment and connection termination phases, as we discussed in Chapter 4. Before discussing RSVP, we need to mention that it is an independent protocol separate from the Integrated Services model. It may be used in other models in the future.

Multicast Trees

RSVP, however, is different from other connection-oriented protocols in that it is based on multicast communication. However, RSVP can also be used for unicasting, because unicasting is just a special case of multicasting with only one member in the multicast group. The reason for this design is to enable RSVP to provide resource reservations for all kinds of traffic including multimedia, which often uses multicasting.

Receiver-Based Reservation

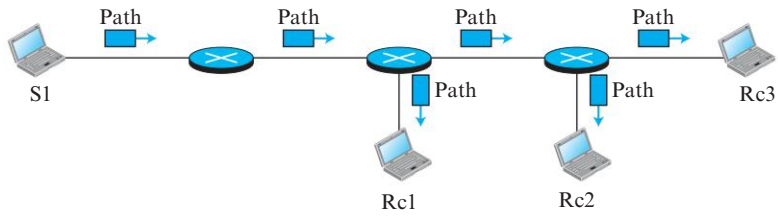
In RSVP, the receivers, not the sender, make the reservation. This strategy matches the other multicasting protocols. For example, in multicast routing protocols, the receivers, not the sender, make a decision to join or leave a multicast group.

RSVP Messages

RSVP has several types of messages. However, for our purposes, we discuss only two of them: *Path* and *Resv.*

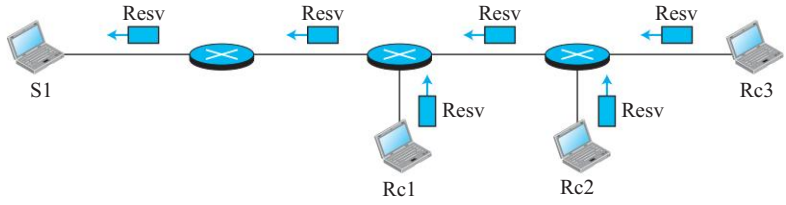
- ***Path Messages.*** Recall that the receivers in a flow make the reservation in RSVP. However, the receivers do not know the path traveled by packets before the reservation is made. The path is needed for the reservation. To solve the problem, RSVP uses *Path* messages. A Path message travels from the sender and reaches all receivers in the multicast path. On the way, a Path message stores the necessary information for the receivers. A Path message is sent in a multicast environment; a new message is created when the path diverges. Figure 8.65 shows path messages.

Figure 8.65 *Path messages*



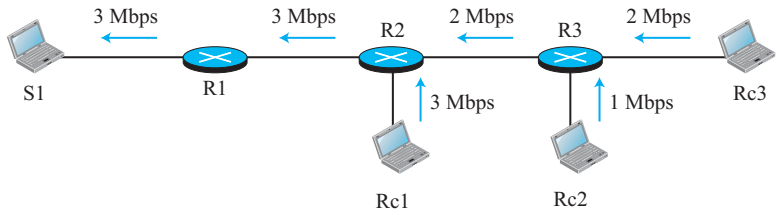
- ***Resv Messages.*** After a receiver has received a Path message, it sends a *Resv* message. The Resv message travels toward the sender (upstream) and makes a resource reservation on the routers that support RSVP. If a router on the path does not support RSVP, it routes the packet based on the best-effort delivery methods we discussed before. Figure 8.66 shows the Resv messages.

Figure 8.66 *Resv messages*



Reservation Merging In RSVP, the resources are not reserved for each receiver in a flow; the reservation is merged. In Figure 8.67, Rc3 requests a 2-Mbps bandwidth while Rc2 requests a 1-Mbps bandwidth. Router R3, which needs to make a bandwidth reservation, merges the two requests. The reservation is made for 2 Mbps, the larger of the two, because a 2-Mbps input reservation can handle both requests. The same situation is true for R2. The reader may ask why Rc2 and Rc3, both belonging to one single flow, request different amounts of bandwidth. The answer is that, in a multimedia environment, different receivers may handle different grades of quality. For example, Rc2 may

Figure 8.67 *Reservation merging*



be able to receive video only at 1 Mbps (lower quality), while Rc3 may want to receive video at 2 Mbps (higher quality).

Reservation Styles When there is more than one flow, the router needs to make a reservation to accommodate all of them. RSVP defines three types of reservation styles: *wild-card filter* (WF), *fixed filter* (FF), and *shared explicit* (SE)

- ***Wild Card Filter Style.*** In this style, the router creates a single reservation for all senders. The reservation is based on the largest request. This type of style is used when the flows from different senders do not occur at the same time.
- ***Fixed Filter Style.*** In this style, the router creates a distinct reservation for each flow. This means that if there are *n* flows, *n* different reservations are made. This type of style is used when there is a high probability that flows from different senders will occur at the same time.
- ***Shared Explicit Style.*** In this style, the router creates a single reservation that can be shared by a set of flows.

Soft State The reservation information (state) stored in every node for a flow needs to be refreshed periodically. This is referred to as a *soft state,* as compared to the *hard state* used in other virtual-circuit protocols such as ATM, where the information about the flow is maintained until it is erased. The default interval for refreshing (softstate reservation) is currently 30 seconds.

Problems with Integrated Services

There are at least two problems with Integrated Services that may prevent its full implementation in the Internet: scalability and service-type limitation.

Scalability

The Integrated Services model requires that each router keep information for each flow. As the Internet is growing every day, this is a serious problem. Keeping information is especially troublesome for core routers because they are primarily designed to switch packets at a high rate and not to process information.

Service-Type Limitation

The Integrated Services model provides only two types of services, guaranteed and control-load. Those opposing this model argue that applications may need more than these two types of services.

8.5.5 Differentiated Services (DiffServ)

In this model, also called **DiffServ,** packets are marked by applications into classes according to their priorities. Routers and switches, using various queuing strategies, route the packets. This model was introduced by the IETF (Internet Engineering Task Force) to handle the shortcomings of Integrated Services. Two fundamental changes were made:

1. The main processing was moved from the core of the network to the edge of the network. This solves the scalability problem. The routers do not have to store information about flows. The applications, or hosts, define the type of service they need each time they send a packet.
2. The per-flow service is changed to per-class service. The router routes the packet based on the class of service defined in the packet, not the flow. This solves the service-type limitation problem. We can define different types of classes based on the needs of applications.

Differentiated Services is a class-based QoS model designed for IP.
In this Model packets are marked by applications according to their priority.

DS Field

In DiffServ, each packet contains a field called the DS field. The value of this field is set at the boundary of the network by the host or the first router designated as the boundary router. IETF proposes to replace the existing ToS (type of service) field in IPv4 or the priority class field in IPv6 with the DS field, as shown in Figure 8.68.

Figure 8.68 *DS field*

The DS field contains two subfields: DSCP and CU. The DSCP (Differentiated Services Code Point) is a 6-bit subfield that defines the **per-hop behavior (PHB).** The 2-bit CU (Currently Unused) subfield is not currently used.

The DiffServ capable node (router) uses the DSCP 6 bits as an index to a table defining the packet-handling mechanism for the current packet being processed.

Per-Hop Behavior

The DiffServ model defines per-hop behaviors (PHBs) for each node that receives a packet. So far three PHBs are defined: DE PHB, EF PHB, and AF PHB.

- ***DE PHB.*** The DE PHB (default PHB) is the same as best-effort delivery, which is compatible with ToS.
- ***EF PHB.*** The EF PHB (expedited forwarding PHB) provides the following services:
  a. Low loss.
  b. Low latency.
  c. Ensured bandwidth.

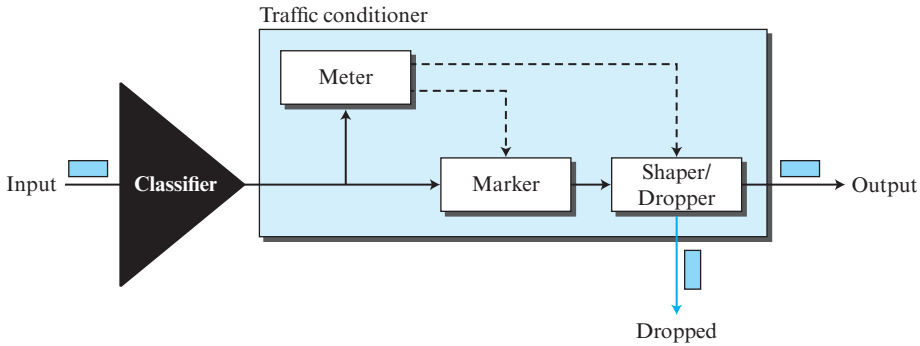
  This is the same as having a virtual connection between the source and destination.

- ***AF PHB.*** The AF PHB (assured forwarding PHB) delivers the packet with a high assurance as long as the class traffic does not exceed the traffic profile of the node. The users of the network need to be aware that some packets may be discarded.

Traffic Conditioner

To implement DiffServ, the DS node uses traffic conditioners such as meters, markers, shapers, and droppers, as shown in Figure 8.69.

Figure 8.69 *Traffic conditioner*



- ***Meters.*** The meter checks to see if the incoming flow matches the negotiated traffic profile. The meter also sends this result to other components. The meter can use several tools such as a token bucket to check the profile.
- ***Marker.*** A marker can remark a packet that is using best-effort delivery (DSCP: 000000) or down-mark a packet based on information received from the meter. Down-marking (lowering the class of the flow) occurs if the flow does not match the profile. A marker does not up-mark a packet (promote the class).
- ***Shaper.*** A shaper uses the information received from the meter to reshape the traffic if it is not compliant with the negotiated profile.
- ***Dropper.*** A dropper, which works as a shaper with no buffer, discards packets if the flow severely violates the negotiated profile.

8.6 END-CHAPTER MATERIALS

8.6.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books and RFCs. The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give some coverage of multimedia: [Com 06], [Tan 03], and [G & W 04].

RFCs

Several RFCs show different updates on topics discussed in this chapter including RFC 2198, RFC 2250, RFC 2326, RFC 2475, RFC 3246, RFC 3550, and RFC 3551.

8.6.2 Key Terms

adaptive DM (ADM)
adaptive DPCM (ADPCM)
arithmetic coding
association
bidirectional frame (B-frame)
delta modulation (DM)
differential PCM (DPCM)
Differentiated Services (DS or DiffServ)
discrete cosine transform (DCT)
first-in, first-out (FIFO) queuing
forward error correction (FEC)
frequency masking
gatekeeper
gateway
H.323
Huffman coding
Integrated Services (IntServ)
intracoded frame (I-frame)
jitter
Joint Photographic Experts Group (JPEG)
leaky bucket algorithm
Lempel-Ziv-Welch (LZW)
linear predictive coding (LPC)
lossless compression
lossy compression
media server
metafile
mixer
Motion Picture Experts Group (MPEG)
MPEG audio layer 3 (MP3)
multihoming service
mutistream service
per-hop behavior (PHB)
perceptual coding
playback buffer
predicted frame (P-frame)
predictive coding (PC)
priority queuing
psychoacoustics
quality of service (QoS)
Real-Time Streaming Protocol (RTSP)
Real-time Transport Control Protocol (RTCP)
Real-time Transport Protocol (RTP)
registrar server
Resource Reservation Protocol (RSVP)
run-length coding
Session Initiation Protocol (SIP)
spatial compression
Stream Control Transmission Protocol (SCTP)
temporal compression
temporal masking
timestamp
token bucket
voice over IP
weighted fair queuing

8.6.3 Summary

We can divide compression into two broad categories: lossless and lossy compression. In lossless compression, the integrity of the data is preserved because compression and decompression algorithms are exact inverses of each other: no part of the data is lost in the process. Lossy compression cannot preserve the accuracy of data, but we gain the benefit of reducing the size of the compressed data.

Audio/video files can be downloaded for future use (streaming stored audio/video) or broadcast to clients over the Internet (streaming live audio/video). The Internet can also be used for live audio/video interaction. Audio and video need to be digitized before being sent over the Internet. We can use a web server, or a web server with a metafile, or a media server, or a media server and RTSP to download a streaming audio/video file.

Real-time data on a packet-switched network requires the preservation of the time relationship between packets of a session. Gaps between consecutive packets at the

receiver cause a phenomenon called *jitter*. Jitter can be controlled through the use of timestamps and a judicious choice of the playback time.

Voice over IP is a real-time interactive audio/video application. The Session Initiation Protocol (SIP) is an application-layer protocol that establishes, manages, and terminates multimedia sessions. H.323 is an ITU standard that allows a telephone connected to a public telephone network to talk to a computer connected to the Internet.

Real-time multimedia traffic requires both UDP and Real-Time Transport Protocol (RTP). RTP handles timestamping, sequencing, and mixing. Real-Time Transport Control Protocol (RTCP) provides flow control, quality of data control, and feedback to the sources. The new transport-layer protocol SCTP has been designed to handle multimedia applications such as voice over IP.

Scheduling, traffic shaping, resource reservation, and admission control are techniques to improve quality of service (QoS). FIFO queuing, priority queuing, and weighted fair queuing are scheduling techniques. Leaky bucket and token bucket are traffic shaping techniques. Integrated Services is a flow-based QoS model designed for IP. The Resource Reservation Protocol (RSVP) is a signaling protocol that helps IP create a flow and makes a resource reservation. Differential Services is a class-based QoS model designed for IP.

8.7 PRACTICE SET

8.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check his/her understanding of the materials before continuing with the practice set.

8.7.2 Questions

Q8-1. Assume that a message is made of four characters (A, B, C, and D) with equal probability of occurrence. Guess what the encoding Huffman table for this message would be. Does encoding here really decrease the number of bits to be sent?

Q8-2. In arithmetic coding, could two different messages be encoded in the same interval? Explain.

Q8-3. In dictionary coding, if there are 60 characters in the message, how many times is the loop in the compression algorithm iterated? Explain.

Q8-4. In dictionary coding, should all dictionary entries that are created in the process be used for encoding or decoding?

Q8-5. In an alphabet with 20 symbols, what is the number of leaves in a Huffman tree?

Q8-6. Is the following code an instantaneous one? Explain.

00 01 10 11 001 011 111

Q8-7. Compare the number of bits transmitted for each PCM and DM sample if the maximum quantized value is

a. 12 **b.** 30 **c.** 50

Q8-8. Answer the following questions about predictive coding:

a. What are slope overload distortion and granular noise distortion in DM coding?

b. Explain how ADM coding solves the above errors.

Q8-9. What is the difference between DM and DPCM?

Q8-10. What is the problem with a speech signal that is compressed using the LPC method?

Q8-11. In predictive coding, differentiate between DM and ADM.

Q8-12. In predictive coding, differentiate between DPCM and ADPCM.

Q8-13. In JPEG, explain why we need to round the result of division in the quantization step.

Q8-14. Explain why the combination of quantization/dequantization steps in JPEG is a lossy process even though we divide each element of the matrix M by the corresponding value in matrix Q when quantizing and multiply it by the same value when dequantizing.

Q8-15. In multimedia communication, assume a sender can encode an image using only JPEG encoding, but the potential receiver can only decode an image if it is encoded in GIF. Can these two entities exchange multimedia data?

Q8-16. When we stream stored audio/video, what is the difference between the first approach (Figure 8.24) and the second approach (Figure 8.25)?

Q8-17. When we stream stored audio/video, what is the difference between the second approach (Figure 8.25) and the third approach (Figure 8.26)?

Q8-18. In the fourth approach to streaming audio/video, what is the role of RTSP?

Q8-19. In transform coding, when a sender transmits the M matrix to a receiver, does the sender need to send the T matrix used in calculation? Explain.

Q8-20. In JPEG, do we need less than 24 bits for each pixel if our image is using one or two primary colors? Explain.

Q8-21. In JPEG, explain why the values of the $Q(m, n)$ in the quantization matrix are not the same. In other words, why is each element in $M(m, n)$ not divided by one fixed value instead of a different value?

Q8-22. Explain why using Q10 gives a better compression ratio but a poorer image quality than using Q90 (*see* Figure 8.18).

Q8-23. In Figure 8.26, can the Web server and media server run on different machines?

Q8-24. In real-time interactive audio/video, what will happen if a packet arrives at the receiver site after the scheduled playback time?

Q8-25. What is the main difference between live audio/video and real-time interactive audio/video?

Q8-26. When we use audio/video on demand, which of these three types of multimedia communication takes place: streaming stored audio/video, streaming live audio/video, or real-time interactive audio/video?

Q8-27. In Figure 8.34, that does the scheme still work if more than one packet is lost?

Q8-28. Assume that we devise a protocol with the packet size so large that it can carry all chunks of a live or real-time multimedia stream in one packet. Do we still need sequence numbers or timestamps for the chunks? Explain.

Q8-29. Assume that we want to send a dataword of two bits. Can the following codewords be used to correct a single bit error?

00000 01011 10101 11110

Q8-30. In Figure 8.33, does interleaving work if more than one packet is lost?

Q8-31. Are encoding and decoding of the multimedia data done by RTP? Explain.

Q8-32. Assume that an image is sent from the source to the destination using 10 RTP packets. Can the first five packets define the encoding as JPEG and the last five packets as GIF?

Q8-33. Explain why RTP cannot be used as a transport-layer protocol without being run on the top of another transport-layer protocol such as UDP.

Q8-34. Both TCP and RTP use sequence numbers. Do sequence numbers in these two protocols play the same role? Explain.

Q8-35. Can UDP without RTP provide an appropriate service for real-time interactive multimedia applications?

Q8-36. If we capture RTP packets, most of the time we see the total size of the RTP header as 12 bytes. Can you explain this?

Q8-37. Can we say UDP plus RTP is the same as TCP?

Q8-38. Why does RTP need the service of another protocol, RTCP, but TCP does not?

Q8-39. UDP does not create a connection. How are different chunks of data, carried in different RTP packets, glued together?

Q8-40. Assume that an application program uses separate audio and video streams during an RTP session. How many SSRCs and CSRCs are used in each RTP packet?

Q8-41. In which situation, a unicast session or a multicast session, can feedback received from an RTCP packet about the session be handled more easily by the sender?

Q8-42. Can the combination of RTP/RTCP and SIP operate in a wireless environment? Explain.

Q8-43. Do some research and find out if SIP can provide the following services provided by modern telephone sets.

a. caller-ID **b.** call-waiting **c.** multiparty calling

Q8-44. In Internet telephony, explain how a call from Alice can be directed to Bob when he can be in his office or at home?

Q8-45. Do you think H.323 is actually the same as SIP? What are the differences? Make a comparison between the two.

Q8-46. Can H.323 also be used for video?

Q8-47. Does SIP need to use the service of RTP? Explain.

Q8-48. We mentioned that SIP is an application-layer program used to provide a signalling mechanism between the caller and the callee. Which party in this communication is the server and which one is the client?

Q8-49. We discuss the use of SIP in this chapter for audio. Is there any drawback to prevent using it for video?

Q8-50. Assume that two parties need to establish IP telephony using the service of RTP. How can they define the two ephemeral port numbers to be used by RTP, one for each direction?

Q8-51. In a bank branch, there is only one teller, but two lines of customers: the business and the regular. The teller will serve the regular customers only if there is no business customer. What type of queuing scheme do we have here? Explain.

Q8-52. In a bank branch, there is only one teller, but two lines of customers: the business and the regular. The teller serves two customers from the business line and one from the line of regular customers. What type of queuing scheme do we have here? Explain.

Q8-53. In a bank branch, there are two tellers. One exclusively serves the business customers; the other serves regular customers. Is this an example of priority queuing? Explain.

Q8-54. In a bank branch, to make the lines of customers shorter, the manager has created three lines. If there is only one teller that serves one customer from each line, what type of queuing scheme do we have here?

8.7.3 Problems

P8-1. In LZW coding, the code "0026163301" is given. Assuming that the alphabet is made of four characters: "A", "B", "C", and "D", decode the message. (*see* Figure 8.3.)

P8-2. Given the message **"AACCCBCCDDAB",** in which the probabilities of symbols are P(A) = 0.50, P(B) = 0.25, P(C) = 0.125 and P(D) = 0.125,

a. encode the data using Huffman coding.

b. find the compression ratio if each original character is represented by 8-bits.

P8-3. Given the following message, find the compressed data using run-length coding:

AAACCCCCCBCCCCDDDDDAAAABBB

P8-4. Given the following message, find the compressed data using the second version of run-length coding with the count expressed as a 4-bit binary number:

10000001000001000000000000010000001

P8-5. In Huffman coding, the following coding table is given:

$A \rightarrow 0$ $B \rightarrow 10$ $C \rightarrow 110$ $D \rightarrow 111$

Show the original message if the code "00110110011110111111010" is received.

P8-6. Given the message **"ACCBCAAB*",** in which the probabilities of symbols are P(A) = 0.4, P(B) = 0.3, P(C) = 0.2 and P(*) = 0.1,

a. find the compressed data using arithmetic coding with a precision of 10 binary digits.

b. find the compression ratio if we use 8-bits to represent a character in the message.

P8-7. In dictionary coding, can you easily find the code if the message is one of the following (the message alphabet has only one character)?

a. "A" **b.** "AA" **c.** "AAA"
d. "AAAA" **e.** "AAAAA" **f.** "AAAAAA"

P8-8. In LZW coding, the message **"AACCCBCCDDAB"** is given.

a. Encode the message. (*see* Figure 8.2.)

b. Find the compression ratio if we use 8-bits to represent a character and four bits to represent a digit (hexadecimal).

P8-9. In predictive coding, we have the following code. Show how we can calculate the reconstructed value (y_n) for each sample if we use delta modulation (DM). We know that $y_0 = 8$ and $\Delta = 6$.

n	1	2	3	4	5	6	7	8	9	10	11
C_n	1	0	0	1	0	1	1	0	0	1	1

P8-10. In predictive coding, assume that we have the following sample (x_n). Show the encoded message sent if we use adaptive DM (ADM). Let $y_0 = 10$, $\Delta_1 = 4$, $M_1 = 1$. Also assume that $M_n = 1.5 \times M_{n-1}$ if $q_n = q_{n-1}$ (no change in q_n) and $M_n = 0.5 \times M_{n-1}$ otherwise.

n	1	2	3	4	5	6	7	8	9	10	11
x_n	13	15	15	17	20	20	18	16	16	17	18

P8-11. In arithmetic coding, assume that we have received the code 100110011. If we know that the alphabet is made of four symbols with the probabilities of P(A) = 0.4, P(B) = 0.3, P(C) = 0.2 and P(*) = 0.1, find the original message.

P8-12. In predictive coding, assume that we have the following sampled (x_n):

n	1	2	3	4	5	6	7	8	9	10	11
x_n	13	24	46	60	45	32	30	40	30	27	20

a. Show the encoded message sent if we use delta modulation (DM). Let $y_0 = 10$ and $\Delta = 8$.

b. From the calculated q_n values, what can you say about the given Δ?

P8-13. In DCT, is the value of $T(m, n)$ in transform coding always between −1 and 1? Explain.

P8-14. In transform coding, show that a receiver that receives an M matrix can create the original p matrix.

P8-15. Calculate the T matrix for DCT when $N = 1$, $N = 2$, $N = 4$, and $N = 8$.

P8-16. Using one-dimensional DCT encoding, calculate the M matrix from the following three p matrices (which are given as row matrices but need to be considered as column matrices). Interpret the result.

$p_1 = [1\ 2\ 3\ 4\ 5\ 6\ 7\ 8]$ $p_2 = [1\ 3\ 5\ 7\ 9\ 11\ 13\ 15]$ $p_3 = [1\ 6\ 11\ 16\ 21\ 26\ 31\ 36]$

P8-17. Assume that we have the following code. Show how we can calculate the reconstructed value (y_n) for each sample if we use ADM. Let $y_0 = 20$, $\Delta_1 = 4$, $M_1 = 1$. Also assume that $M_n = 1.5 \times M_{n-1}$ if $q_n = q_{n-1}$ (no change in q_n) and $M_n = 0.5 \times M_{n-1}$ otherwise.

n	1	2	3	4	5	6	7	8	9	10	11
C_n	1	1	1	0	0	1	1	0	0	1	1

P8-18. In one-dimensional DCT, if $N = 1$, the matrix transformation is changed to simple multiplication. In other words, $M = T \times p$, in which T, p, and M are numbers (scalar value) instead of matrices. What is the value of T in this case?

P8-19. In the interleaving approach to FEC, assume each packet contains 10 samples from a sampled piece of music. Instead of loading the first packet with the first 10 samples, the second packet with the second 10 samples, and so on, the sender loads the first packet with the odd-numbered samples of the first 20 samples, the second packet with the even-numbered samples of the first 20 samples, and so on. The receiver reorders the samples and plays them. Now assume that the third packet is lost in transmission. What will be missed at the receiver site?

P8-20. Assume that we want to send a dataword of 2-bits using FEC based on the Hamming distance. Show how the following list of datawords/codewords can automatically correct up to a one-bit error in transmission.

$00 \rightarrow 00000$ $01 \rightarrow 01011$ $10 \rightarrow 10101$ $11 \rightarrow 11110$

P8-21. Assume that an image uses a palette of size 8 out of the table used by JPEG (GIF uses the same strategy, but the size of the palette is 256), with the combination of the following colors with the indicated level of intensities.

Red: 0 and 7 Blue: 0 and 5 Green: 0 and 4

Show the palette for this situation and answer the following questions:

a. How many bits are sent for each pixel?

b. What are the bits sent for the following pixels: red, blue, green, black, white, and magenta (red and blue, but no green)?

P8-22. In the first approach to streaming stored audio/video (Figure 8.24), assume that we need to listen to a compressed song of 4 megabytes (a typical situation). If our connection to the Internet is via a 56 Kbps modem, how long will we need to wait before the song can be started (downloading time)?

P8-23. In Figure 8.31, what is the amount of data in the playback buffer at each of the following times?

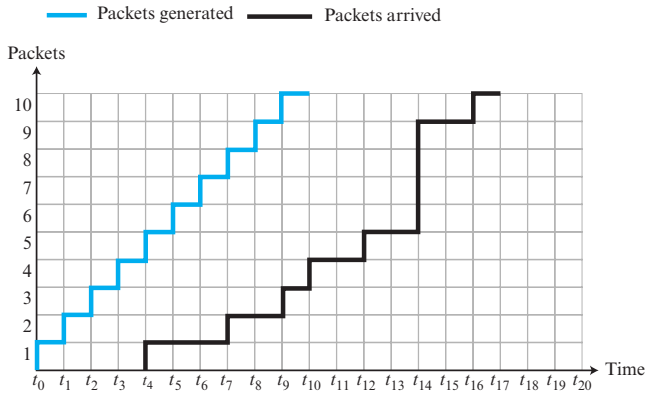
a. 00:00:17 **b.** 00:00:20 **c.** 00:00:25 **d.** 00:00:30

P8-24. Figure 8.70 shows the generated and the arrival time for ten audio packets. Answer the following questions:

a. If we start our player at t_8, which packets cannot be played?

b. If we start our player at t_9, which packets cannot be played?

Figure 8.70 *Problem 8-24*



P8-25. Assume that we need to create codewords that can automatically correct a one-bit error. What should be the number of redundant bits (r) given the number of bits in the dataword (k)? Remember that the codeword needs to be $n = k + r$ bits, called C(n,k). After finding the relationship, find the number of bits in r if k is 1, 2, 5, 50, or 1000.

P8-26. In the previous problem, we tried to find the number of bits to be added to a dataword to correct a single-bit error. If we need to correct more than one bit, the number of redundant bits increases. What should be the number of redundant bits (r) to automatically correct one or two bits (not necessarily contingent) in a dataword of size k? After finding the relationship, find the number of bits in r if k is 1, 2, 5, 50, or 1000.

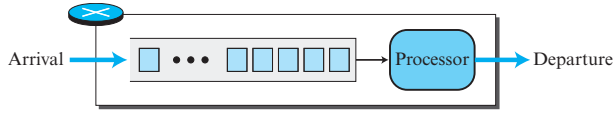
P8-27. Using the ideas in the previous two problems, we can create a general formula for correcting any number of errors (m) in a codeword of size (n). Develop such a formula. Use the combination of n objects taking x objects at a time.

P8-28. In Figure 8.34, assume that we have 100 packets. We have created two sets of packets with high and low resolutions. Each high-resolution packet carries on average 700 bits. Each low-resolution packet carries on an average 400 bits. How many extra bits are we sending in this scheme for the sake of FEC? What is the percentage of overhead?

P8-29. Explain why TCP, as a byte-oriented stream protocol, is not suitable for applications such as live or real-time multimedia streaming.

P8-30. Figure 8.71 shows a router using FIFO queuing at the input port.

Figure 8.71 *Problem 8-30*



The arrival and required service times for seven packets are shown below; t_i means that the packet has arrived or departed i ms after a reference time. The values of required service times are also shown in ms. We assume the transmission time is negligible.

Packets	1	2	3	4	5	6	7
Arrival time	t_0	t_1	t_2	t_4	t_5	t_6	t_7
Required service time	1	1	3	2	2	3	1

a. Using time lines, show the arrival time, the process duration, and the departure time for each packet. Also show the contents of the queue at the beginning of each millisecond.

b. For each packet, find the time spent in the router and the departure delay with respect to the previously departed packet.

c. If all packets belong to the same application program, determine whether the router creates jitter for the packets.

P8-31. Given an RTP packet with the first eight hexadecimal digits as $(86032132)_{16}$, answer the following questions:

a. What is the version of the RTP protocol?

b. Is there any padding for security?

c. Is there any extension header?

d. How many contributors are defined in the packet?

e. What is the type of payload carried by the RTP packet?

f. What is the total header size in bytes?

P8-32. In a real-time multimedia communication, assume that we have one sender and ten receivers. If the sender is sending multimedia data at 1 Mbps, how many RTCP packets can be sent by the sender and each receiver in a second? Assume that the system allocates 80 percent of the RTCP bandwidth to the receivers and 20 percent to the sender. The average size of each RTCP packet is 1000-bits.

P8-33. In Figure 8.61, assume that the weight in each class is 4, 2, and 1. The packets in the top queue are labeled A, in the middle queue B, and in the bottom queue C. Show the list of packets transmitted in each of the following situations:

a. Each queue has a large number of packets.

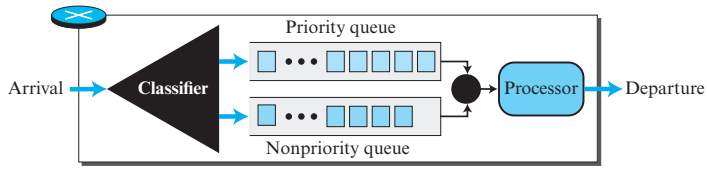
b. The numbers of packets in queues, from top to bottom, are 10, 4, and 0.

c. The numbers of packets in queues, from top to bottom, are 0, 5, and 10.

P8-34. In a leaky bucket used to control liquid flow, how many gallons of liquid are left in the bucket if the output rate is 5 gal/min, there is an input burst of 100 gal/min for 12 s, and there is no input for 48 s?

P8-35. Figure 8.72 shows a router using priority queuing at the input port.
The arrival and required service times (transmission time is negligible) for 10 packets are shown below; t_i means that the packet has arrived i ms after a reference time. The values of required service times are also shown in ms. The packets with higher priorities are packets 1, 2, 3, 4, 7, and 9 (shown in color); the other packets are packets with lower priorities.

Figure 8.72 *Problem 8-33*



Packets	1	2	3	4	5	6	7	8	9	10
Arrival time	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9
Required service time	2	2	2	2	1	1	2	1	2	1

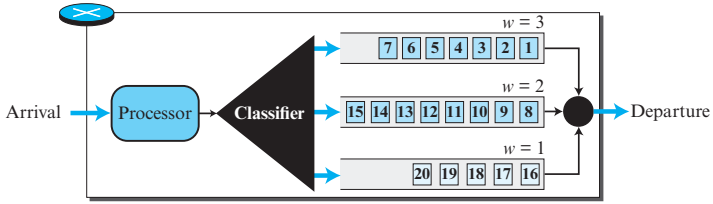
a. Using time lines, show the arrival time, the processing duration, and the departure time for each packet. Also show the contents of the priority queue (Q1) and the nonpriority queue (Q2) at each millisecond.

b. For each packet belonging to the priority class, find the time spent in the router and the departure delay with respect to the previously departed packet. Find if the router creates jitter for this class.

c. For each packet belonging to the nonpriority class, find the time spent in the router and the departure delay with respect to the previously departed packet. Determine whether the router creates jitter for this class.

P8-36. To regulate its output flow, a router implements a weighted queueing scheme with three queues at the output port. The packets are classified and stored in one of these queues before being transmitted. The weights assigned to queues are w = 3, w = 2, and w = 1 (3/6, 2/6, and 1/6). The contents of each queue at time t_0 are shown in Figure 8.73. Assume that the packets are all of the same size and that transmission time for each is 1 μs.

Figure 8.73 *Problem 8-36*



a. Using a time line, show the departure time for each packet.

b. Show the contents of the queues after 5, 10, 15, and 20 μs.

c. Find the departure delay of each packet with respect to the previous packet in the class w = 3. Has queuing created jitter in this class?

d. Find the departure delay of each packet with respect to the previous packet in the class w = 2. Has queuing created jitter in this class?

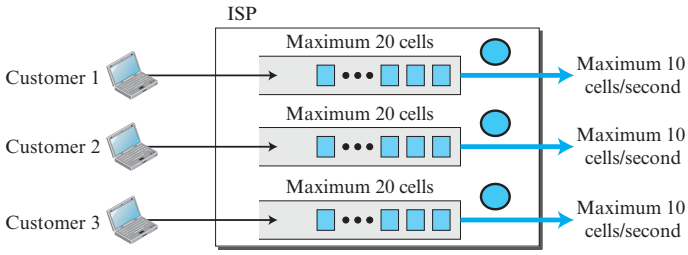
e. Find the departure delay of each packet with respect to the previous packet in the class w = 1. Has queuing created jitter in this class?

P8-37. An output interface in a switch is designed using the leaky bucket algorithm to send 8000 bytes/s (tick). If the following frames are received in sequence, show the frames that are sent during each second.

- Frames 1, 2, 3, 4: 4000 bytes each
- Frames 5, 6, 7: 3200 bytes each
- Frames 8, 9: 400 bytes each
- Frames 10, 11, 12: 2000 bytes each

P8-38. Assume that an ISP uses three leaky buckets to regulate data received from three customers for transmitting to the Internet. The customers send fixed-size packets (cells). The ISP sends 10 cells per second for each customer and the maximum burst size of 20 cells per second. Each leaky bucket is implemented as a FIFO queue (of size 20) and a timer that extracts one cell from the queue and sends it every 1/10 of a second. (*see* Figure 8.74.)

Figure 8.74 *Problem 8-38*



a. Show the customer rate and the contents of the queue for the first customer, which sends 5 cells per second for the first 7 seconds and 15 cells per second for the next 9 seconds.

b. Do the same for the second customer, which sends 15 cells per second for the first 4 seconds and 5 cells per second for the next 14 seconds.

c. Do the same for the third customer, which sends no cells for the first two seconds, 20 cells for the next two seconds, and repeats the pattern four times.

P8-39. Assume that fixed-sized packets arrive at a router with a rate of three packets per second. Show how the router can use the leaky bucket algorithm to send out only two packets per second. What is the problem with this approach?

P8-40. Assume that a router receives packets of size 400-bits every 100 ms, which means with the data rate of 4 Kbps. Show how we can change the output data rate to less than 1 Kbps by using a leaky bucket algorithm.

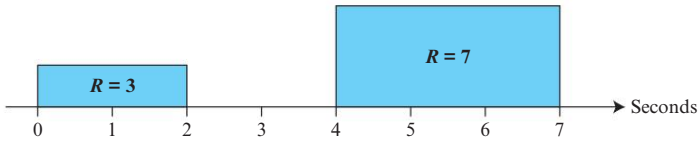
P8-41. In a switch using the token bucket algorithm, tokens are added to the bucket at a rate of $r = 5$ tokens/second. The capacity of the token bucket is $c = 10$. The switch has a buffer that can hold only eight packets (for the sake of example). The packets arrive at the switch at the rate of R packets/second. Assume that the packets are all the same size and need the same amount of time for pro-

cessing. If at time zero the bucket is empty, show the contents of the bucket and the queue in each of the following cases and interpret the result.

a. $R = 5$ **b.** $R = 3$ **c.** $R = 7$

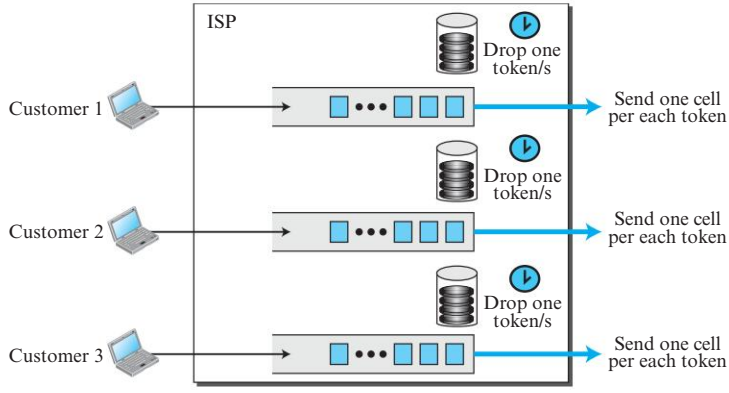
P8-42. To understand how the token bucket algorithm can give credit to the sender that does not use its rate allocation for a while but wants to use it later, let us repeat the previous problem with $r = 3$ and $c = 10$, but assume that the sender uses a variable rate, as shown in Figure 8.75. The sender sends only three packets per second for the first two seconds, sends no packets for the next two seconds, and sends seven packets for the next three seconds. The sender is allowed to send five packets per second, but, since it does not use this right fully during the first four seconds, it can send more in the next three seconds. Show the contents of the token bucket and the buffer for each second to prove this fact.

Figure 8.75 *Problem 8-42*



P8-43. Assume that the ISP in the Problem P8-38 decided to use token buckets (of capacity $c = 20$ and rate $r = 10$) instead of leaky buckets to give credit to the customer that does not send cells for a while but needs to send some bursts later. Each token bucket is implemented by a very large queue for each customer (no packet drop), a bucket that holds the token and the timer that regulates dropping tokens in the bucket. (*see* Figure 8.76.)

Figure 8.76 *Problem 8-43*



a. Show the customer rate, the contents of the queue, and the contents of the bucket for the first customer, which sends five cells per second for the first seven seconds and 15 cells per second for the next nine seconds.

b. Do the same for the second customer, which sends 15 cells per second for the first four seconds and five cells per second for the next 14 seconds.

c. Do the same for the third customer, which sends no cells for the first two seconds, 20 cells for the next two seconds, and repeats the pattern four times.

8.8 SIMULATION EXPERIMENTS

8.8.1 Applets

We have created some Java applets to show some of the main concepts discussed in this chapter. It is strongly recommended that students activate these applets on the book website and carefully examine the protocols in action.

8.8.2 Lab Assignments

In this chapter, we use the Wireshark to capture and study multimedia packets such as images, audio, and video packets.

Lab8-1. In this lab, we need to examine the contents of a multimedia packet to find the encoding used by the sender.

Lab8-2. In this lab, we need to examine the contents of multimedia packet to find which transport-layer protocol is used for communication.

8.9 PROGRAMMING ASSIGNMENTS

Write the source code for, compile, and test the following programs in one of the programming languages of your choice:

Prg8-1. Modify the general UDP client-server program in Chapter 2 (in C) or the one in Chapter 11 (Java) to include RTP. The program needs to use RTP library (in C) or the RTP Java class to send and receive JPEG images.

Prg8-2. A program to simulate a leaky bucket.

Prg8-3. A program to simulate a token bucket.

Prg8-4. A program for coding and decoding the first version of the run-length compression method.

Prg8-5. A program for coding and decoding the second version of the run-length compression method.

Prg8-6. A program simulating Table 8.1 (LZW encoding).

Prg8-7. A program simulating Table 8.2 (LZW decoding).

Prg8-8. A program simulating Table 8.4 (arithmetic encoding).

Prg8-9. A program simulating Table 8.5 (arithmetic decoding).

Prg8-10. A program that reads a two-dimensional matrix of size $N \times N$ and writes the values using zigzag ordering described in the chapter.

Prg8-11. A program that finds the DCT transform of a one-dimensional matrix. Implement matrix multiplication.

Prg8-12. A program that finds the DCT transform of a two-dimensional matrix. Implement matrix multiplication.